

# A Theory of Programming

**Christian Rinderknecht**

`Christian.Rinderknecht@nomadic-labs.com`

*Nomadic Labs (Paris)*

20 October 2018

Tezos Masterclass

- This series of lectures proposes a stepwise introduction to **functional programming** and the analyses to write reliable and efficient software.
- The theoretical background of functional languages is formal logic, where mathematical functions are the main building block.
- This close link with formal logic implies that the meaning of functional programs can often be made very precise.
- It is a good framework to get a sense of **static analyses**, and, as such, it stands a prominent place in the field of formal methods and **compiler construction**.

- In a later lecture, we will learn a specific programming language called **OCaml**, enabling you to put in practice what you have learnt.
- OCaml is used for the core development of the **Tezos blockchain**.
- Most textbooks and courses on OCaml propose a traditional approach: basic concepts, data structures, algorithms.
- Here, we aim more at illustrating the kind of software **Tezos** uses OCaml for, in particular, compiler construction.
- This first lecture is not specific to OCaml and is theoretical because it introduces many concepts that are used in functional programming and compiler construction.

# String rewriting

- Let us consider a string of white and black beads, like  $\circ \bullet \bullet \bullet \circ \circ \bullet$ , and a one-player game whose unique move consists in replacing two adjacent beads with only one according to the rules

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \bullet \xrightarrow{\beta} \bullet \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

- The rules  $\alpha$ ,  $\beta$  and  $\gamma$  make up a simple **string-rewriting system**.
- Rules  $\alpha$  and  $\beta$  can be conceived as:  
*A black bead absorbs the white bead next to it.*
- The goal of this game is to end up with as few beads as possible, so our example may lead to the **rewrites**

$$\circ \bullet \bullet \boxed{\bullet \circ} \circ \bullet \xrightarrow{\alpha} \circ \bullet \bullet \boxed{\bullet \circ} \bullet \xrightarrow{\alpha} \boxed{\circ \bullet} \bullet \bullet \bullet \xrightarrow{\beta} \bullet \bullet \bullet \boxed{\bullet \bullet} \xrightarrow{\gamma} \bullet \boxed{\bullet \circ} \xrightarrow{\alpha} \boxed{\bullet \bullet} \xrightarrow{\gamma} \circ$$

where the part of the string to be rewritten next is framed.

- Other compositions of the rules lead to the same result  $\circ$  as well. Some others bring all-white or all-black strings, like  $\circ\circ$  and  $\bullet$ .
- Strings that can not be further rewritten, or **reduced**, are called **normal forms**.
- We may wonder whether all strings have a normal form, and, if so, if it is unique and, furthermore, if it is either all-white or all-black.
- First, let us note that the system is **terminating**, that is, there is no infinite chain of rewrites, because the number of beads strictly decreases in all the rules.
- This is not a necessary condition in general, for instance,  $\circ \bullet \xrightarrow{\beta} \bullet \circ \circ$  would preserve termination because the composition  $\beta\alpha\alpha$  would be equivalent to the original rule  $\beta$ .
- In particular, this means that any string has a normal form.

- Second, notice how the parity of the number of black beads is **invariant** through each rule and how there is no rewrite rule for two adjacent white beads.
- Therefore, if there are  $2p$  initial black beads, then composing rules  $\alpha$  and  $\beta$  lead to an all-black string, which can be reduced by applying rule  $\gamma$  to contiguous pairs of beads into an all-white string made of  $p$  beads.
- Otherwise, the same all-black string can be reduced by applying alternatively  $\gamma$  and  $\beta$  on the left end or  $\gamma$  and  $\alpha$  on the right end, yielding  $\circ$ .
- Similarly, if there is an initial odd number of black beads, we always end up with one black bead.

- Consequently, normal forms are not unique.
- A simple example is

$$\circ\circ \xleftarrow{\gamma} \bullet\bullet\circ \xrightarrow{\alpha} \bullet\bullet \xrightarrow{\gamma} \circ$$

- Systems where normal forms are unique are **confluent**.
- If we think of a rewrite step as an elementary computation, it becomes clear that confluence is a desirable property for carrying out computations, as we would expect, for instance, an arithmetic expression to have the same value no matter the order of the small steps.
- A systemic order of rule application is called a **reduction strategy**.

- We can obtain confluence by adding the rule  $\delta$ :

$$\bullet \circ \xrightarrow{\alpha} \bullet$$

$$\circ \bullet \xrightarrow{\beta} \bullet$$

$$\bullet \bullet \xrightarrow{\gamma} \circ$$

$$\circ \circ \xrightarrow{\delta} \circ$$

the result of the game is always one bead, whose colour depends on the original parity of the black beads as before.

- To see why, let us consider first that two non-overlapping parts of a string can be rewritten in parallel, so they are not a concern.
- The interesting cases occur when two applications of rules (maybe of the same rule) lead to different strings because they do overlap. For instance,

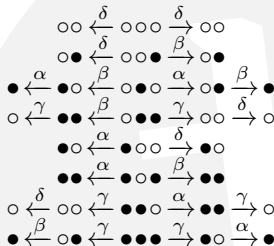
$$\circ \circ \xleftarrow{\gamma} \bullet \bullet \circ \xrightarrow{\alpha} \bullet \bullet$$

- The important point is that  $\circ \circ$  and  $\bullet \bullet$  can be rewritten into  $\circ$  at the next step by  $\delta$  and  $\gamma$ , respectively.



# Critical pairs

- In general, what matters is that all pairs of strings resulting from the application of overlapping rules, called **critical pairs**, can be rewritten to the same string: they are **joinable**.
- In our example, all interactions occur on substrings made of three beads, so we must examine eight cases:



- In all the cases, the divergences are joinable in one step at most.

- If all critical pairs are joinable after a divergence in one ore more rewrites, the system is **locally confluent**.
- Moreover, if the system is also terminating, then **every** string has exactly **one** normal form (a strong property entailing confluence).
- Sometimes, as seen previously, a non-confluent system can be completed to become confluent, via a **completion procedure**.
- In the case of terminating systems, one of such semi-decision procedures is the Knuth-Bendix algorithm.

- The system we defined is **ground**, that is, it involves no variables.
- These allow a finite system to denote an infinite number of ground rules if variables range over infinite sets, or simply, they reduce the size of rewrite systems.
- For instance, our continued example is equivalent to

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ x \xrightarrow{\beta+\delta} x \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

- If we accept multiple occurrences of a variable on the left-hand side of a rule, a so-called **non left-linear rule**, we can further decrease the size of the system:

$$xx \xrightarrow{\gamma+\delta} \circ \qquad xy \xrightarrow{\alpha+\beta} \bullet$$

- Notice that there is now an implicit order over the rules: the rule  $\gamma + \delta$  must be examined first for a **match** with a part of the current string, because it is included in the second (set  $x = y$  in  $\alpha + \beta$ ).
- Non-linear systems are equational theories due the implicit equality on substructures, which is complex: we will limit ourselves to **linear systems**.
- In general, the order in which the rules are written on the page is their **logical order**.
- Moreover, let us remark that the system does not specify that  $x$  must either be  $\circ$  or  $\bullet$ : this should be stated nevertheless somewhere else.
- Generally speaking, this means that the **type** of the variables has to be defined elsewhere or inferred from the variable uses.
- We will return to that topic about OCaml.

- More general are the **term-rewriting systems**, where a **term** is a mathematical object built on tuples, integers and variables.
- Let us consider the following totally ordered system:

$$(0, m) \xrightarrow{1} m$$

$$(n, m) \xrightarrow{2} (n-1, n \times m)$$

$$n \xrightarrow{3} (n, 1)$$

- Arithmetic operators  $(-)$  and  $(\cdot)$  are defined out of the system and  $m$  and  $n$  are variables denoting natural numbers.
- Would the rules not be ordered as they are actually laid out, the second rule would match any pair. Instead, it can be assumed that  $n \neq 0$  when matching with it.
- The last rule is to be tried last, as it matches all terms.

- We can easily see that all the compositions of rewrites starting with a natural number  $n$  end with the factorial of  $n$ :

$$n \xrightarrow{3} (n,1) \xrightarrow{2} \dots \xrightarrow{2} (0,n!) \xrightarrow{1} n!, \quad \text{for } n \in \mathbb{N}.$$

- Let us note  $(\xrightarrow{n})$  the composition of  $(\rightarrow)$  repeated  $n-1$  times:

$$\begin{aligned} (\xrightarrow{1}) &:= (\rightarrow) \\ (\xrightarrow{n+1}) &:= (\rightarrow) \circ (\xrightarrow{n}), \quad \text{with } n > 0. \end{aligned}$$

- The **transitive closure** of  $(\rightarrow)$  is defined as

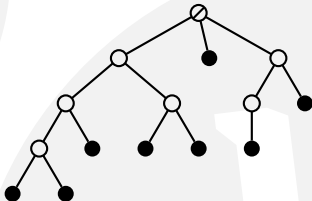
$$(\twoheadrightarrow) := \bigcup_{i > 0} (\xrightarrow{i}).$$

- The factorial function coincides with the transitive closure of  $(\rightarrow)$ :  $n \twoheadrightarrow n!$ .
- Let  $(\xrightarrow{*})$  be the reflexive-transitive closure of  $(\rightarrow)$ , that is,

$$(\xrightarrow{*}) := (=) \cup (\twoheadrightarrow).$$

- A confluent system defines a **partial function**.
- If the system is also terminating, we get a **function**.
- We can name functions: for example,  $c(1, d(n))$  is a term constructed with **function names**  $c$  and  $d$ , as well as variable  $n$ .
- A tuple tagged with a function name, like  $f(x, y)$ , is called a **function call**.
- The components of the tuples are then called **arguments**, for example  $d(n)$  is the second argument of the call  $c(1, d(n))$ .
- It is possible for a function call to hold no arguments, like  $d()$ .
- We restrict the left-hand sides of rules to be function calls.

- The topological understanding of a function call or a tuple is the finite **tree**.
- A tree is a hierarchical layout of information, for example:



- The disks are called **nodes** and the segments which connect two nodes are called **edges**. The topmost node (with a diameter) is called the **root** and the bottommost ones (●) are called the **leaves**. All nodes except the leaves are seen to downwardly connect to some other nodes, called **children**. Upwardly, each node but the root is connected to another node, called its **parent**.



- **Trees are pervasive** in informatics, especially in functional programming, but not only.
- Structured documents, like books, articles, and web pages, are composed of chapters, sections, paragraphs, figures, appendices, indices, etc.
- The occurrences of these components are mutually constrained; for instance, it is understood that a section is part of a chapter and that appendices are located at the end of a document.
- This hierarchical layout is meant to facilitate reading, and it supports the search for specific items of information.
- When considering computer systems, these data must be uniformly encoded by means of a formal language.

## Terms as trees

- Trees can be used to depict terms as follows. A function call is a tree whose root is the function name and the children are the trees denoting the arguments.
- A tuple can be considered as having an invisible function name represented by a node with a period (.) in the tree, in which case the components of the tuple are its children. For example,



is the tree corresponding to the term  $f((g(0), (x, 1)), (), g(y))$ .

- Note that tuples, except the empty tuple, are not represented in the tree, since they encode the structure itself, which is already laid out. The number of arguments of a function is called **arity**.

- For programming, we only consider confluent systems because they define partial functions.
- To have only one reduction chain per term to the normal form, we may also enforce that rules are ordered.
- We also enforce that normal forms must be **values**, that is, terms without function calls or only with calls to functions not occurring on the left-hand sides.
- In other words, irreducible calls of defined functions are not allowed in values.
- Systems with these constraints make up a **functional language**.
- A program here is a term, and evaluating it means to rewrite the term
  1. to a value,
  2. or to end in error with a normal form that is not a value,
  3. or to never terminate.

- We do not require by construction that all functional programs terminate, although that is a desirable property, and we speak nevertheless of “functions” instead of “partial functions” (which would be technically correct).
- If we enforce termination, we loose **Turing-completeness**: we cannot program all computable functions.
- The termination of rewrite systems and, in particular, functional programs, is an **undecidable problem** in general.
- Nevertheless, there exists a set of well-known **decision criteria**, which we will use when programming recursive functions in OCaml.

- If a function call contains at least one other function call, there is a choice as to which one to reduce first.
- One reduction strategy, named **call-by-value**, consists in always reducing the arguments before the call itself.
- Unfortunately, call-by-value enables otherwise terminating rewrites to not terminate. For instance, let us consider

$$f(x) \xrightarrow{\alpha} 0 \quad g() \xrightarrow{\beta} g()$$

We have  $f(g()) \xrightarrow{\alpha} 0$  but  $f(g()) \xrightarrow{\beta} f(g()) \xrightarrow{\beta} \dots$

- Despite the loss of expressiveness, we shall retain in this lecture the call by value, as it is the strategy of OCaml and it is easy to follow.
- In general, the order in which the arguments of a function call are evaluated is not specified.

- Another reason to choose call-by-value is that it allows us to restrict the shape of the left-hand sides, called **patterns**, to one, outermost function call.
- For instance, we can then disallow, for being useless, a rule like

$$\text{plus}(x, \text{plus}(y, z)) \rightarrow \text{plus}(\text{plus}(x, y), z).$$

because, following call-by-value, the argument  $\text{plus}(y, z)$  must be evaluated before the outermost call, therefore it cannot be part of any pattern.

- Most functional languages allow **higher-order function** definitions, whereas standard term-rewriting systems do not. For example:

$$f(g,0) \rightarrow 1 \quad f(g,n) \rightarrow n \times g(g,n-1) \quad \text{fact}(n) \rightarrow f(f,n)$$

where  $n \in \mathbb{N}$ . Note that these two definitions are **not** recursive, yet the function **fact** is the factorial function.

- A function definition is **recursive** if the name of the definition occurs on at least one right-hand side.
- An adequate theoretical framework to understand higher-order functions is  **$\lambda$ -calculus**.
- In fact,  $\lambda$ -calculus features prominently in the semantics of programming languages, even not functional ones.
- For didactic purposes, we began with rewrite systems because they offer implicit pattern matching (that is, rule selection).

- We show how to express linear data structures in a purely functional language and how to compute with them.
- The simplest of those structures is the **stack**.
- A stack can be thought of as a finite series of items that can only be accessed sequentially from one end, called the **top**, following the analogy with a stack of material objects.



- To define a stack in a functional language, one uses calls to undefined functions to model all the possible shapes a stack can take.
- Here, a stack can be either empty or not.
- Let  $\text{nil}()$  be the irreducible call that denotes an empty stack (the function  $\text{nil}$  is undefined).
- If not empty, the stack then must contain a topmost (first) item, and a **substack**, that is, the stack made of all the items but the first.
- Let  $\text{cons}(x, s)$  denote the non-empty stack with the item  $x$  on top and the **immediate substack**  $s$ .

- The previous definition is intuitive, but not formal yet.
- Data structures are usually defined **inductively**.
- Let  $\mathcal{T}$  be the set of all possible terms and  $\mathcal{S} \subset \mathcal{T}$  be the set of all stacks.
- Formally,  $\mathcal{S}$  can be defined by **induction** as the smallest set such that
  1.  $\text{nil}() \in \mathcal{S}$ ;
  2. if  $x \in \mathcal{T}$  and  $s \in \mathcal{S}$ , then  $\text{cons}(x, s) \in \mathcal{S}$ .
- We take the smallest set because we do not want in  $\mathcal{S}$  terms that were not constructed by the inductive rules (this is called the **smallest fixed point** of the inductive relation).

- For all  $s, t \in \mathcal{S}$ , let us consider the functional program

$$\begin{aligned} \text{cat}(\text{nil}(), t) &\xrightarrow{\alpha} t \\ \text{cat}(\text{cons}(x, s), t) &\xrightarrow{\beta} \text{cons}(x, \text{cat}(s, t)) \end{aligned}$$

- This is a functional program suitable for call-by-value reduction strategy, because the function calls in the patterns (i.e., left-hand sides) are irreducible.
- The right-hand sides are stacks, so a call to “cat” is a stack.
- Rule  $\alpha$  says that, if the first stack is empty, the value is the second.
- In rule  $\beta$ , the first stack is not empty, and its top item is the top of the final stack ( $x$ ), the immediate substack ( $s$ ) being used in a call to “cat” with the second stack ( $t$ ) unchanged.

## Appending a stack

- This function appends the second stack at the bottom of a **copy** of the first. (The function name “cat” stands for “catenation”, and is slightly better than “app”).
- We could also say that this function appends a copy of the first stack on top of the other. But this is not the same as pushing the first stack on top of the other, like

$\text{cat}(s, t) \rightarrow \text{cons}(s, t)$       Wrong!

- This is **not** appending: here, the stack  $s$  becomes an item of the resulting stack, but we want only the **contents** of  $s$  to be pushed on top of  $t$ , not their container.
- Analogy: You may think of an empty stack as a empty cardboard box, open topside, and every item pushed in it is like a book that you put on the topmost book, or at the bottom of the box if this is the first book.

## Appending a stack

- Let us set the abbreviations
  - $[] := \text{nil}()$ ,
  - $x :: s := \text{cons}(x, s)$ ,after the convention of the programming language OCaml.
- We can further abbreviate
  - $x_1 :: x_2 :: \dots :: x_n :: s := \text{cons}(x_1, \text{cons}(x_2, \dots \text{cons}(x_n, s)))$
  - $[x] := x :: []$
  - $[x_1; x_2, \dots; x_n] := x_1 :: x_2 :: x_3 :: \dots :: x_n :: []$ .
- Our system now becomes a bit more legible:

$$\begin{aligned} \text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t) \end{aligned}$$

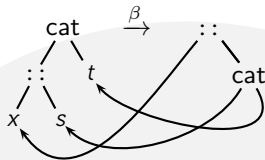
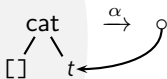
- Let us compute  $\text{cat}([1;2],[3])$ .
- We need to find the first rule that is matched by this call.
- Remember that the arguments must be values before you do that.
- Intuitively, this means that we compare the call with the patterns, that is, the left-hand sides of the rules, in order.
- Here, rule  $\alpha$  is not matched because  $[1;2] \neq []$ .
- Rule  $\beta$  is matched because  $[1;2] = 1 :: [2]$  and equals the sub-pattern  $x :: s$  if we assign  $x := 1$  and  $s := [2]$ . Also, trivially,  $t := [3]$  works.
- After the rule  $\beta$  is selected, the assignments are used to replace the variables in the right-hand sides.
- The whole evaluation is

$$\text{cat}([1;2],[3]) \xrightarrow{\beta} 1 :: \text{cat}([2],[3]) \xrightarrow{\beta} 1 :: 2 :: \text{cat}([], [3]) \xrightarrow{\alpha} [1;2;3].$$

- Depending on the context, we may use the arborescent depiction of terms to bring to the fore certain aspects of a computation.
- For example, it may be interesting to show how parts of the output (the right-hand side) are actually **shared** with the input (the left-hand side).
- In other words, how much (new) data is created by a given rule.
- This concept supposes that terms reside in some sort of space and that they can be referred to from different other terms.
- This abstract space serves as a model of an interpreter's memory, called the **heap**.

# Data sharing

- Consider for instance



which is the same definition of "cat" as given above:

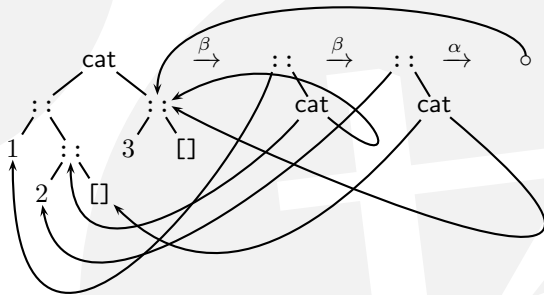
$$\text{cat}([], t) \xrightarrow{\alpha} t$$

$$\text{cat}(x::s, t) \xrightarrow{\beta} x::\text{cat}(s, t)$$

- The arrows on certain edges denote data sharing.
- When trees are used to visualise terms, they are called **abstract syntax trees** (AST).
- When some trees share subtrees, the whole **forest** (set of trees) is called a **directed acyclic graph** (DAG).

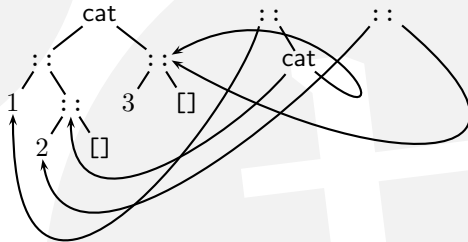


- The evaluation of  $\text{cat}([1;2],[3])$  is:

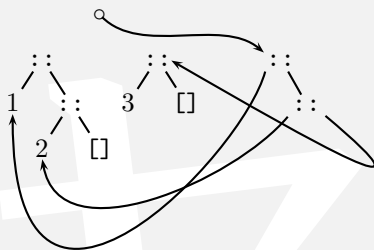
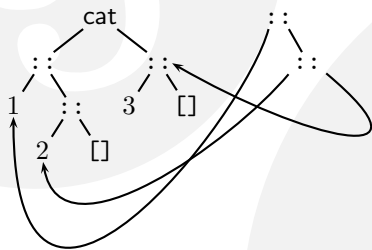


# Data sharing and evaluation

- Note that each right-hand side shows only the rewritten call, not the whole term.
- Therefore, to reconstruct the value, we need to work leftwards and replace the rewritten call by the right-hand side:



- We do not show the calls that have been replaced by their values.



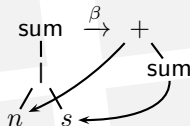
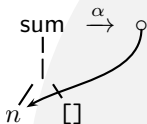
- We can see that the value of the second argument, [3], is shared with the value of the call, as well as the integers 1 and 2.
- We saw how rewrites accumulated rightwards until the result had to be reconstructed by replacing leftwards the right-hand sides.
- This dynamic is that of a stack, but a stack holding suspended function calls (that is, awaiting their values) and references to ASTs in the heap.
- This is the **call stack**.
- The top of the stack (here, the rightmost abstract syntax tree) is the next term to be rewritten, unless the call on the left-hand side is the original call, in which case the top is its value.

- In the abstract syntax trees (AST), the nodes with the calls that are replaced by their values have been immediately removed.
- In fact, they could have been removed later, in any order.
- The process which identifies and discards useless ASTs is called the **garbage collector** (GC).
- A term is useless if it cannot be accessed from anywhere in the call stack.
- Contrary to the heap, which can be thought of as an unordered set of trees (a forest of DAGs), the call stack is an ordered set that can be efficiently extended or reduced.
- The garbage collector can be conceived as a process interleaved with the process of evaluation.
- Like evaluators of functional programs, garbage collectors may also have different strategies (as to when dispose of useless data).

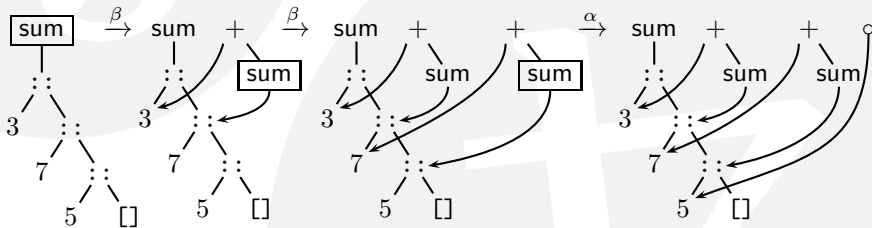
- Let us define a function that sums integers in a non-empty list:

$$\text{sum}([n]) \xrightarrow{\alpha} n \quad \text{sum}(n::s) \xrightarrow{\beta} n + \text{sum}(s)$$

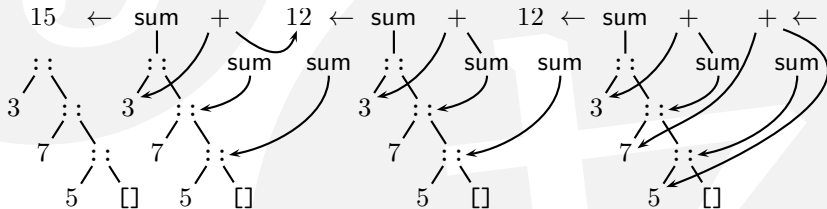
- The view of the system with maximum sharing is:



The first phase of the evaluation of  $\text{sum}([3;7;5])$  is:

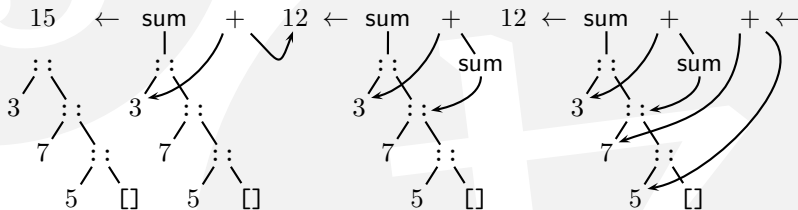


The second phase (replacing the pending calls with their values) is





The same, with a garbage collector that collects after each replacement:



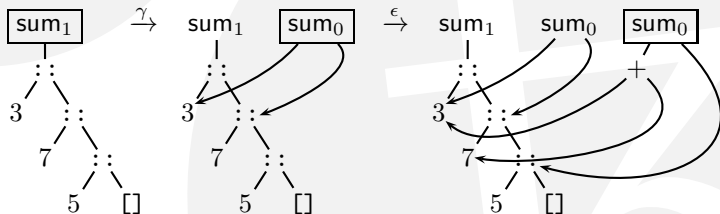
- Let us consider an alternate version of `sum`:

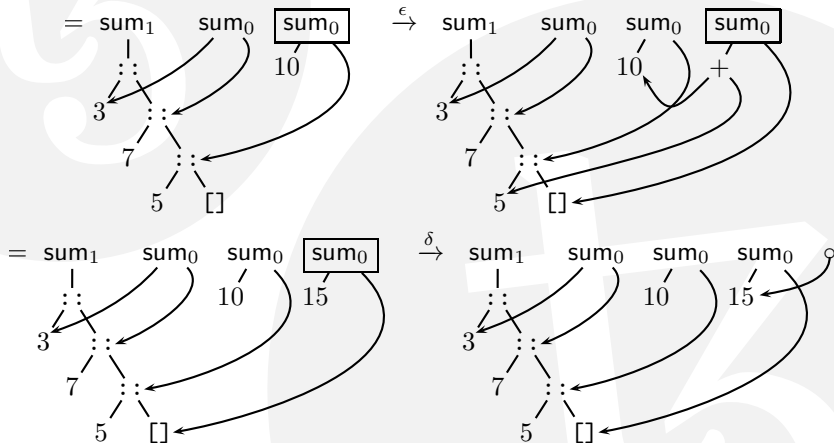
$$\text{sum}_1(n :: t) \xrightarrow{\gamma} \text{sum}_0(n, t).$$

$$\text{sum}_0(n, []) \xrightarrow{\delta} n$$

$$\text{sum}_0(n, m :: s) \xrightarrow{\epsilon} \text{sum}_0(n + m, s).$$

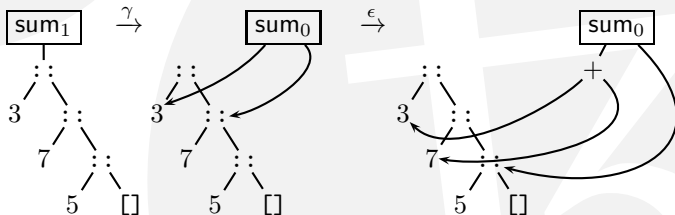
- For `sum1([3;7;5])` we have the evaluation



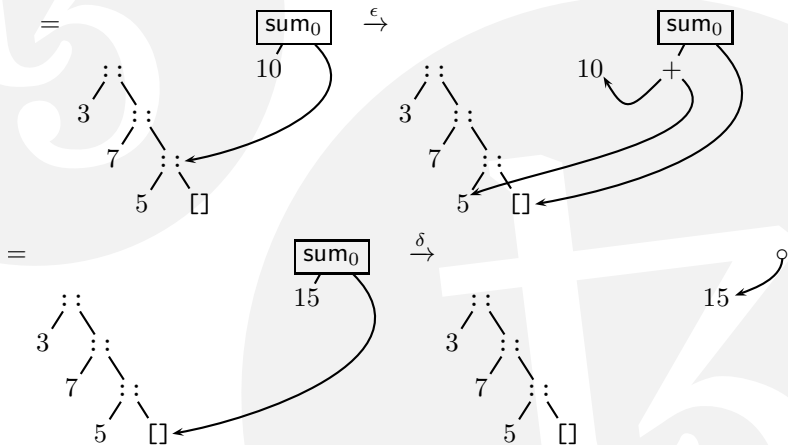


# Tail call optimisation

- Notice that, in the case of  $\text{sum}_0$ , we found the result (15) at the end of the first phase (of growing the call stack).
- Therefore, there is no need for a second phase, which replaces the pending calls by their values: we can discard all those calls at once after the first phase.
- Or, to see it even more clearly, let us rewrite the same first phase and assume that the collector reclaims immediately the useless pending calls:



# Tail call optimisation



- As we see now, we can keep the size of the call stack constant, if we reclaim the top at each new rewrite.
- This optimisation is called **tail call optimisation**.
- Intuitively, a call in **tail position** is a call whose value is the value of the previously rewritten call.
- For example, the call  $f(x)$  in  $1 + f(x)$  is not in tail position, because its value needs to be incremented.
- Most compilers of functional languages detect and implement that optimisation that saves memory.
- Even the C compiler of GCC.